

Day 1: Control Flow, Loops, and Patterns

Goal: Master decision-making structures (if-else, switch) and iteration (for, while, do-while), culminating in building interactive menus and geometric patterns.

Warm-up: Making Decisions

1. The Strict Leap Year Validator

- **Task:** Ask the user to input a year. Determine if it is a leap year and print the result.
- **Focus:** Chaining logical AND (&&) and OR (||) operators. Remember the real leap year rule: A year is a leap year if it is divisible by 4, *except* if it is divisible by 100, *unless* it is also divisible by 400.

2. The Grade Categorizer

- **Task:** Take a student's percentage as input. Use an if-else if-else ladder to assign a grade: A (90+), B (80-89), C (70-79), D (60-69), or F (<60). Handle invalid inputs (like entering 105 or -5) by printing an error message.

3. The Vowel/Consonant Switch

- **Task:** Accept a single alphabetical character from the user. Use a switch statement to check if it's a vowel (a, e, i, o, u) or a consonant.
- **Challenge:** Make sure your switch handles both uppercase and lowercase inputs elegantly (hint: you can stack case statements, or convert the string to lowercase first).

Intermediate: The Power of Repetition

4. The Multiplication Table Generator

- **Task:** Ask the user for an integer. Use a for loop to print its multiplication table from $x \times 1$ up to $x \times 10$ in a neatly formatted list.

5. Factorial Calculator

- **Task:** Calculate the factorial of a user-provided number (e.g., $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$). Use a while loop for this.
- **Focus:** Managing an accumulator variable and preventing infinite loops.

6. The Stubborn Password Prompt

- **Task:** Hardcode a secret password in a String variable. Use a do-while loop to continuously ask the user to "Enter the password". The loop should only terminate when they guess it correctly, followed by a "Access Granted" message.
- **Focus:** Understanding why do-while is perfect for scenarios where the code *must* execute at least once before checking a condition.

Advanced: Complex Logic & Number Games

7. Fibonacci Sequence Generator

- **Task:** Ask the user how many terms of the Fibonacci sequence they want to see. Print that many terms. (The sequence starts with 0 and 1, and every subsequent number is the sum of the previous two: 0, 1, 1, 2, 3, 5, 8...).

8. Prime Number Checker

- **Task:** Ask the user for a positive integer. Determine if it is a prime number (divisible only by 1 and itself).
- **Challenge:** Do not loop all the way up to the number. For optimal performance, you only need to check for divisors up to the square root of the number ($\sqrt{n}$). Use `Math.sqrt()` and a `break` statement if a divisor is found.

9. Reverse & Palindrome Check

- **Task:** Take a multi-digit integer from the user (like 12321). Using a while loop, modulo (%), and division (/), reverse the number. Check if the original number matches the reversed number. If they match, print that it is a Palindrome.

Synthesis: Nested Loops & Interactive Systems

10. Right-Angled Triangle Pattern

- **Task:** Ask the user for a row count. Use a nested for loop (a loop inside a loop) to print a right-angled triangle using asterisks.
- **Output Example (Rows = 4):**

Plaintext

```
*
**
***
****
```

11. The Centered Pyramid Pattern

- **Task:** Similar to problem 10, but this time print a centered pyramid. You will need to manage spaces in addition to the asterisks.
- **Output Example (Rows = 4):**

Plaintext

```
  *
 ***
*****
*****
```

12. Mini-Project: The Console ATM Menu

- **Task:** Build a text-based ATM interface.
- **Requirements:**
 - Initialize a starting balance of \$1000.
 - Use a while(true) loop to wrap a switch statement that displays a menu: 1. Check Balance, 2. Deposit, 3. Withdraw, 4. Exit.
 - If Withdraw is chosen, check if the amount exceeds the balance (print "Insufficient funds" if it does).
 - Keep returning the user to the main menu until they select 4. Exit, which should trigger a break or change the loop condition to terminate the program.